



PSOC™ Edge: Advanced voice and audio development

Customer training workshop (CTW)

August 2026



Table of contents

1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

Prerequisites

Recommended training prerequisites

- Introduction to PSOC™ Edge and ModusToolbox™
 - For an introduction to PSOC™ MCUs, including a getting started guide to ModusToolbox™, go to: <https://www.infineon.com/product-information/psocdeveloper>
 - For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, go to: [PSOC™ Edge E84 Training Collection](#)

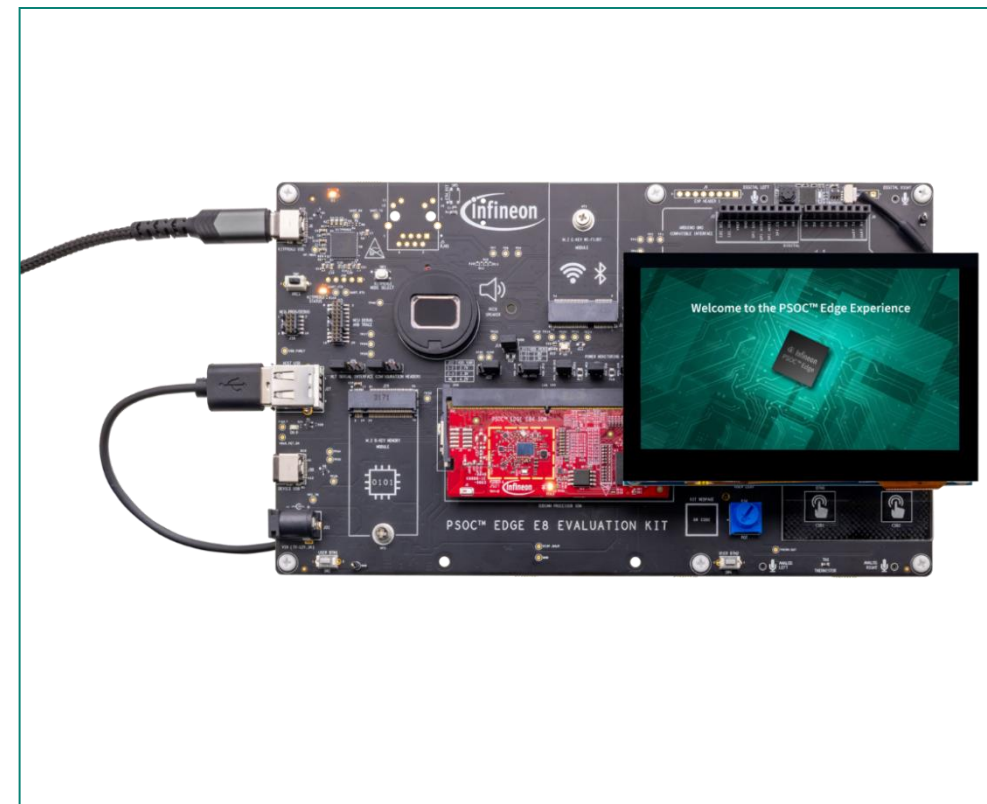
Prerequisites tools

Development tools for all labs (see training manual for links)

- ModusToolbox™ v3.8
- Eclipse IDE for ModusToolbox™ v2026.3.0
- Edge Protect Security Suite v1.6
- ModusToolbox™ programming tools v1.8.1
- KIT_PSE84_EVAL_EPC2 BSP v1.0.0
- Terminal emulator
- KIT_PSE84_EVAL

Requirements for audio labs

- DEEPCRAFT™ Audio Enhancement Tech Pack v1.2.0
- LLVM for Arm® v19.1.5
- Audacity



Note: Other tool versions and hardware can be used with minimal or no modifications to this training manual.

Audio signal processing pipeline on PSOC™ Edge MCUs

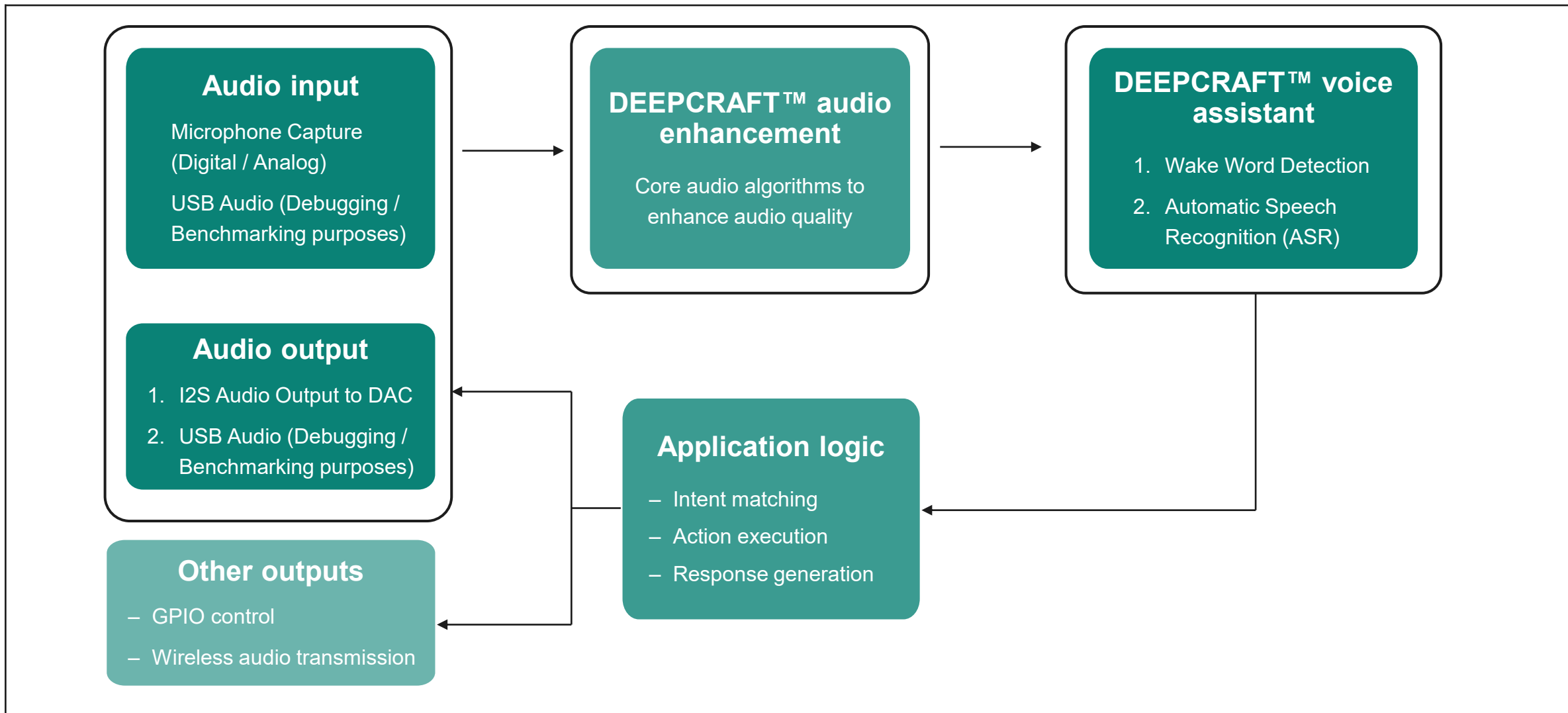
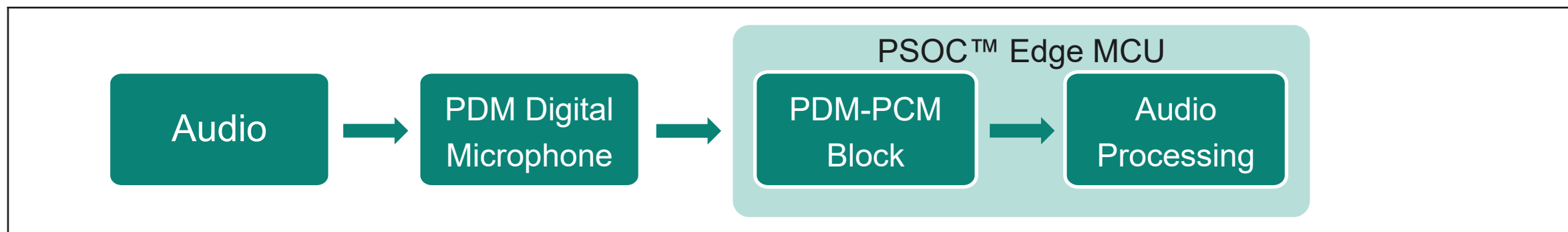


Table of contents

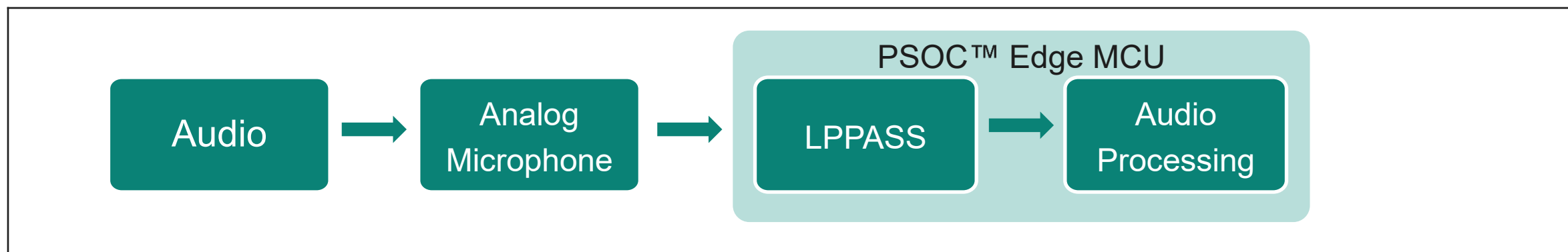
1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

Audio inputs on PSOC™ Edge MCU

Digital microphone input

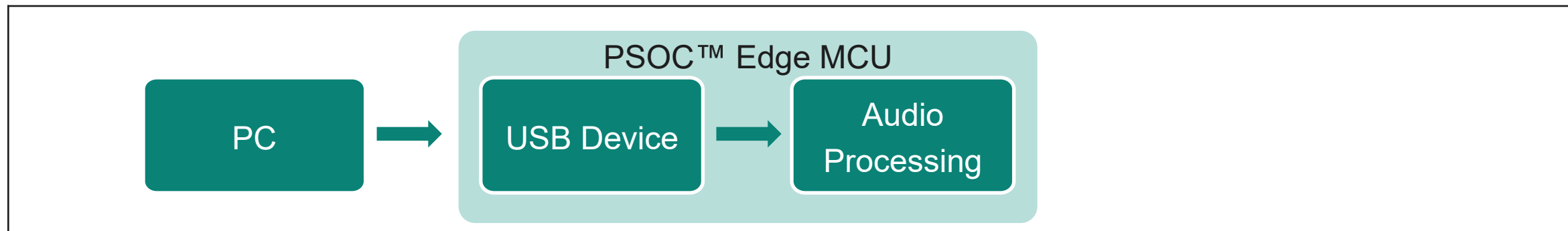


Analog microphone input



Audio inputs on PSoC™ Edge MCU (contd.)

USB audio input – USB audio class

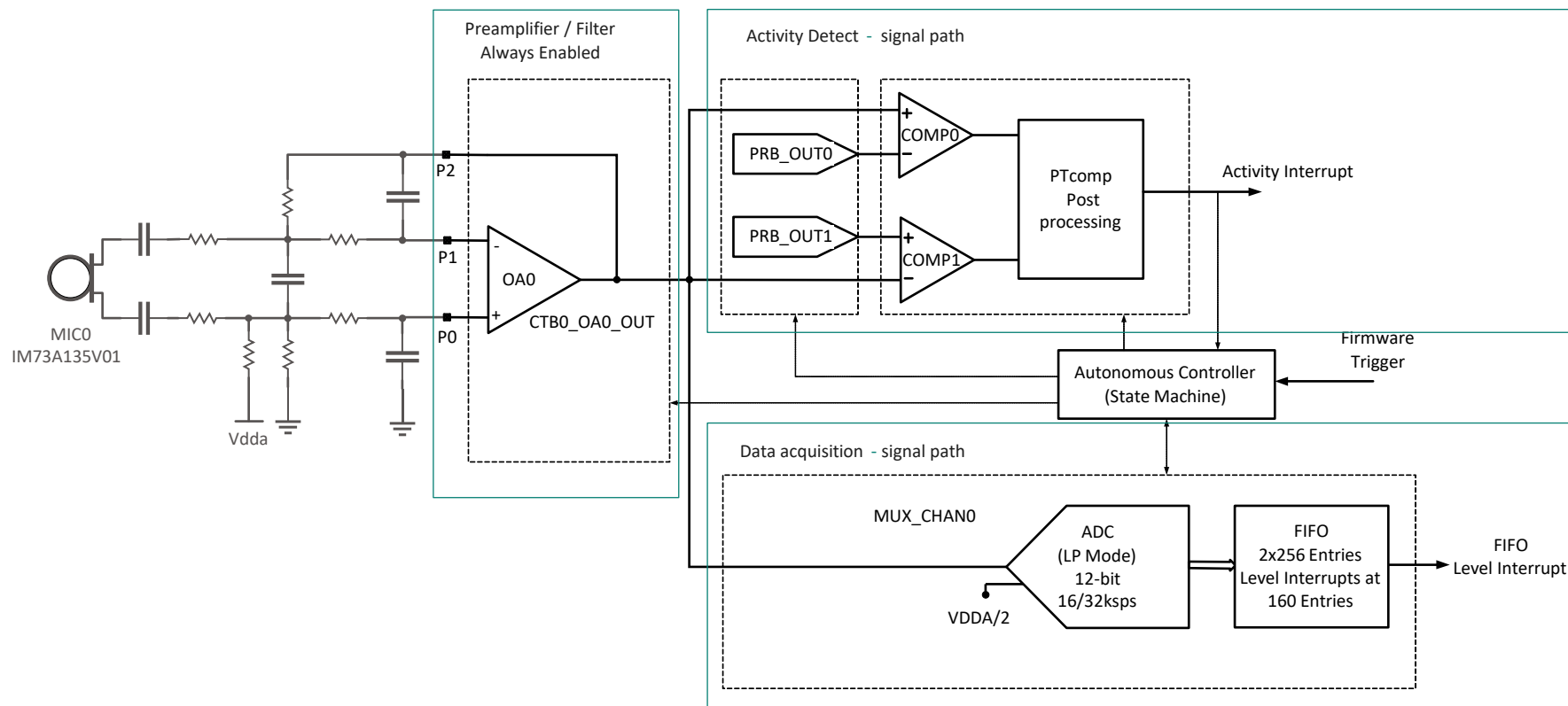


Audio input methods – Comparison

Feature	Analog microphone (MIC)	PDM digital microphone (MIC)
Microphone output	Continuous analog signal	High-speed serial (1-bit digital stream)
Interface	Autonomous analog front-end (op-amp, pre-amplifier/filter, ADC, activity detect)	PDM-PCM block
Bit depth	Typically 12-bit to 20-bit numerical resolution with hardware accumulation	1-bit PDM stream converted to 16-bit or 24-bit by PDM-PCM block
SNR	Medium to very good	Excellent
Power consumption	Ultra-low to medium (depends on power mode and SNR)	Medium to High (depends on clock frequency)
Microphone cost	Low to medium	Medium to High
Best	Low-power / combined low-power and high-quality voice	High-quality audio, voice

Note: USB audio input is typically used for debugging and benchmarking purposes unless the application requires USB Audio.

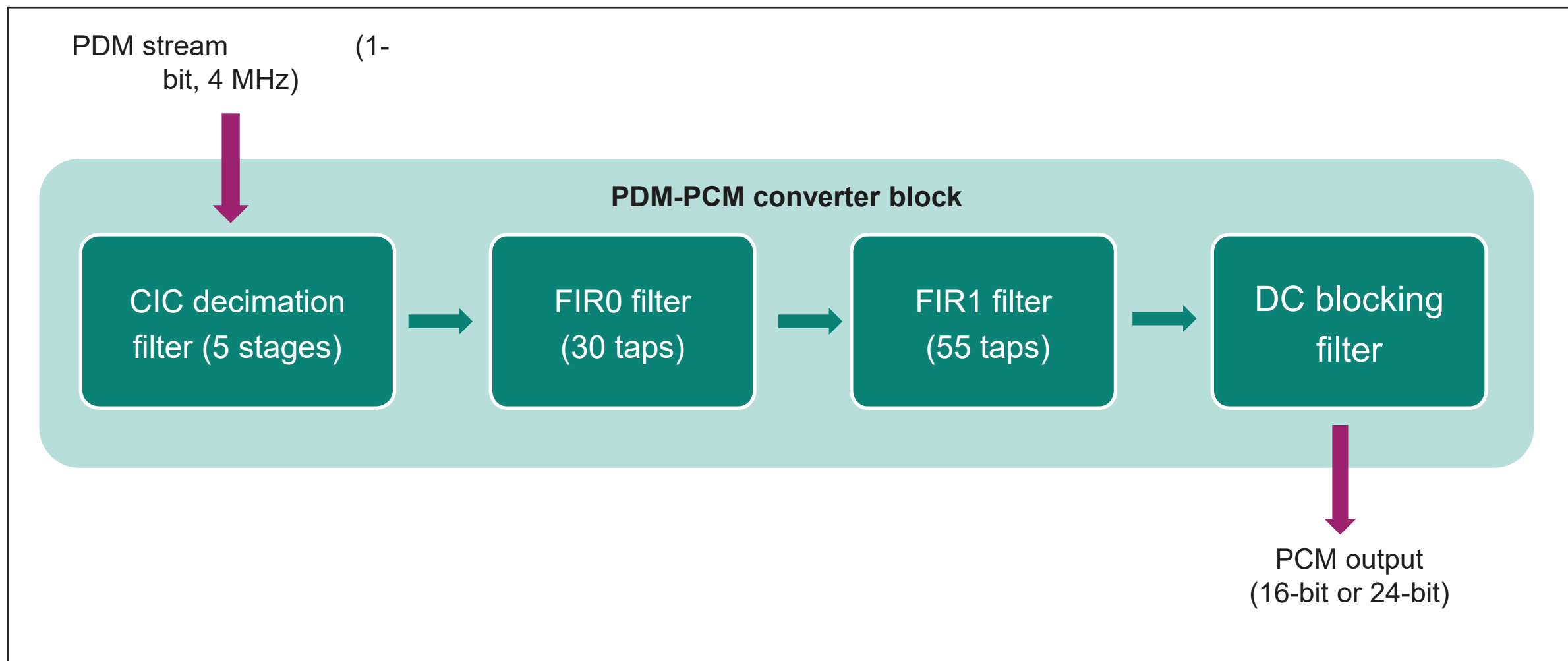
Analog Microphone (AMIC) interface



- Deep Sleep capable ultra-low power front-end with activity detect
- 12-bit, ~50 dB SNR in Deep Sleep mode, ideal for low-power ML, <200 μ W for always-on activity detect, <300 μ W for voice acquisition (MCU power consumption)
- 16- to 20-bit, ~70 dB SNR in Active mode, ideal for voice-call applications

PDM-PCM converter block

Architecture



16-bit vs 24-bit configuration

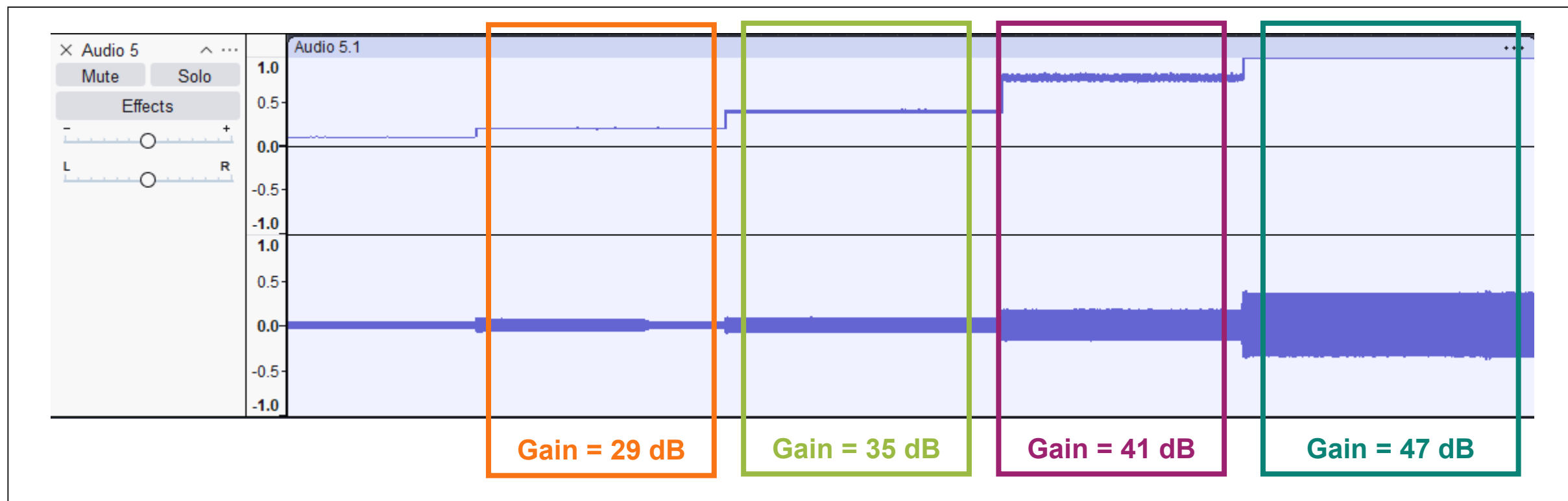
Audio clipping on PSOC™ Edge Kits

- With default 16-bit PCM configuration: A gain value >29 dB could lead to premature clipping of audio data in one channel
- Reason
 - The IM7x series of XENSIV™ MEMS microphones used on kits has unequal DC offset in left and right channels, causing one of the channels to clip first due to a larger DC offset at larger gain settings
 - The gain in the PDM-PCM block is applied through the FIR filters
 - As the gain is applied before the DC blocking filter, audio clipping occurs prematurely
 - Range for 16-bit Configuration: $\pm 32,768$
 - Usable gain before clipping: ~29 dB
- Solution
 - Use 24-bit configuration: Range: $\pm 8,388,608$, Usable gain: + 50 dB (effective range improves significantly)
 - If the application operates on 16-bit PCM, the 24-bit PCM output from the PDM-PCM block can be scaled to 16-bit output in the application, which is demonstrated in Lab1
- Conclusion
 - If the application requires no gain control or a limited gain range → 16-bit configuration is sufficient
 - If gain control is required → 24-bit configuration is needed

16-bit vs 24-bit configuration (contd.)

Audio capture with 16-bit PDM configuration

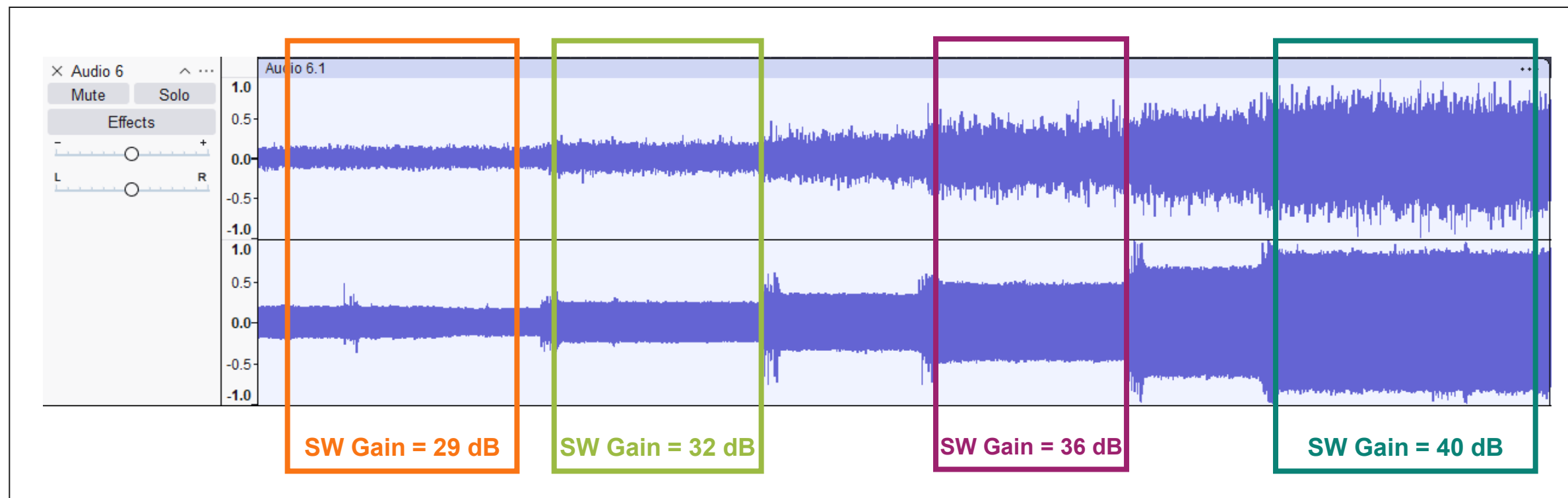
- As the gain increases, the offsets (baseline) keep rising on the top channel
- The audio clips on the positive axis for a gain of 47 dB in this case. If the audio source is near the microphone, the clipping could start at 35 dB onwards



16-bit vs 24-bit configuration (contd.)

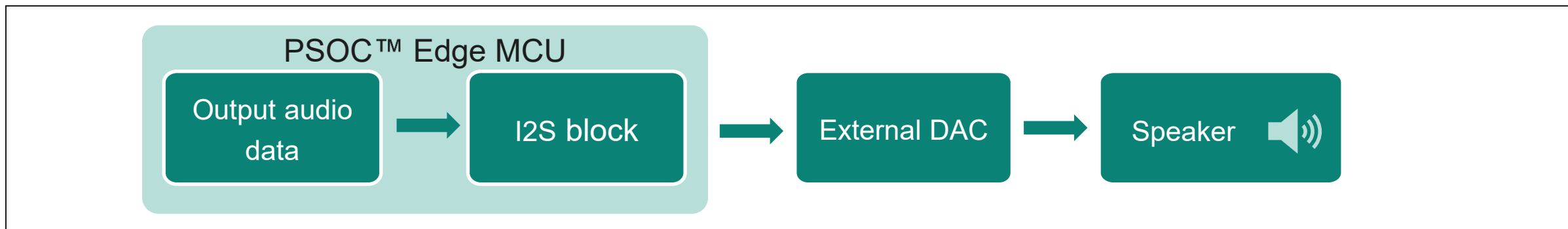
Audio capture with 24-bit PDM configuration

- As the gain increases, the audio is not getting clipped with the 24-bit configuration
- Note that the indicated “SW Gain” is a software gain that’s over and above the modified FIR filter gains in the PDM block
- This effectively increases the headroom / usable gain ranges for the PDM microphones

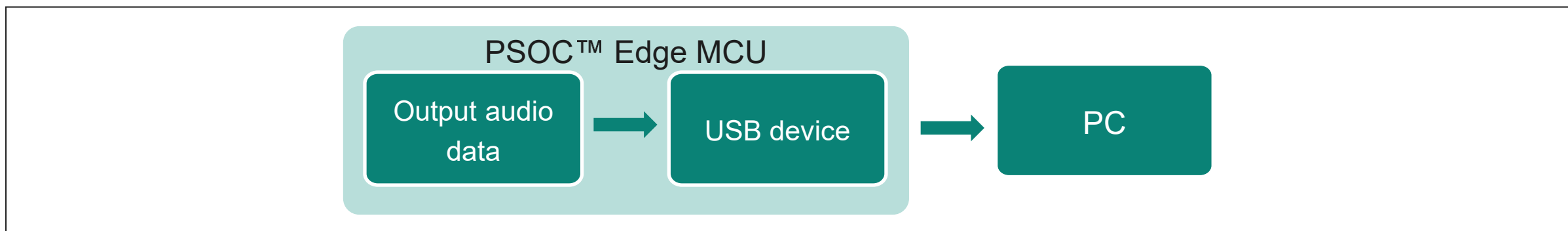


Audio output methods on PSOC™ Edge MCU

I2S audio output played using external DAC



USB audio output: USB audio class



Audio input / output on PSOC™ Edge MCU – Code examples

– PDM to PCM

- Simplest example to interface with PDM microphones
- Demonstrates usage of the PDM-PCM block on the Arm® Cortex®-M33 CPU
- The volume of captured audio is depicted over the serial terminal

– I2S audio

- Simplest example to play audio on the speaker connected to an external DAC interfaced over the I2S block
- Demonstrates usage of I2S block on Arm® Cortex®-M33 CPU
- Predefined audio (stored in flash) is played out of the speaker upon the user button press

– PDM to I2S

- Demonstrates routing of PDM audio data to I2S on CM33 CPU
- Audio data from the PDM mic is recorded upon button press, stored in an internal buffer, and then played out over I2S

Audio input / output on PSOC™ Edge MCU – Code examples (contd.)

- Audio recorder using an emUS
 - Demonstrates how to implement an audio recorder application using USB Audio Class (UAC)
 - Segger's emUSB-device is used as the USB stack
 - PSOC™ Edge will enumerate as a USB microphone on the PC

- Audio playback using emUSB-device
 - Superset application using USB Audio Class (UAC) that implements:
 - Audio playback device
 - Microphone/recording device
 - Consumer control device for volume control
 - Segger's emUSB-device is used as the USB stack

Table of contents

1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

Lab 1: Understanding PDM microphone configuration (16-bit vs 24-bit)

What will you learn ?

- How to use ModusToolbox™ and Audacity to debug audio applications
- Demonstrate the dynamic range limitations of a 16-bit PDM-PCM configuration in the presence of microphone DC offset
- Observe audio clipping in real-time as digital gain is increased
- How to implement the 24-bit PDM-PCM configuration to achieve a higher usable dynamic range

Table of contents

1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

DEEPCRAFT™ Audio Enhancement

Overview

DEEPCRAFT™ Audio Enhancement (AE) is a comprehensive audio signal processing solution that enables developers to customize audio pipelines for various use cases, including:

- Speech enhancement for voice applications
- Noise and echo reduction
- Spatial audio processing (beamforming)
- Real-time tuning and optimization

Typical use cases

- Teleconferencing echo cancellation
- Smart speaker noise suppression
- Conference room audio enhancement
- Voice command recognition preprocessing

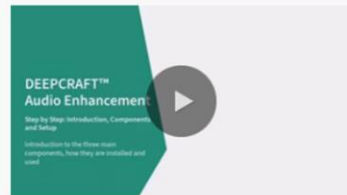
Getting started

To get started with the audio enhancement, see the **DEEPCRAFT™ Audio Enhancement development flow** section in the [PSOC™ Developer Journey Trainings](#).

1. Start your Audio Enhancement design

This video explains the basic hardware and software of Audio Enhancement and the basics for including it in your audio designs.

[Open resource](#)

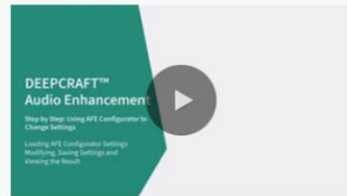


DEEPCRAFT™ Audio Enhancement
Step 1: Introduction, Components and Setup
Introduction to the three main components, how they are installed and used.

[Open in media player](#)

2. Using AFE Configurator to change settings

Loading AFE Configurator settings, modifying, saving settings and viewing the result.



DEEPCRAFT™ Audio Enhancement
Step 2: Using AFE Configurator to Change Settings
Loading AFE Configurator Settings, Modifying, Saving Settings and Viewing the Result.

[Open in media player](#)

3. Recording and inspecting audio from DEEPCRAFT(tm) Audio Enhancement

Recording audio using AFE Configurator. Playing back and viewing the result in Audacity.



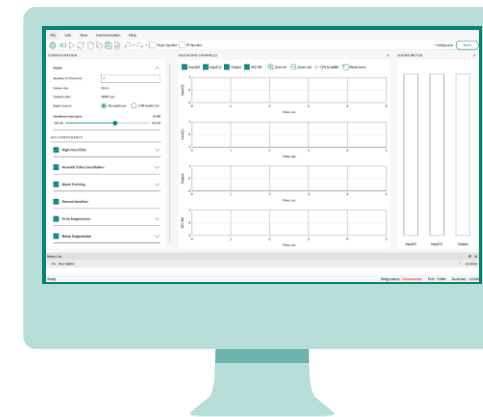
DEEPCRAFT™ Audio Enhancement
Step 3: Using AFE Configurator to Capture and Analyze Audio
Recording audio using AFE Configurator, Playing back and viewing the result in Audacity.

[Open in media player](#)

DEEPCRAFT™ Audio Enhancement package

DEEPCRAFT™ Audio Enhancement consists of three main components:

- **Audio Front End (AFE) configurator**
 - PC Tool
 - Configures the audio pipeline and parameters of various audio processing blocks within AFE middleware
 - Generates .c/.h files used by the AFE middleware
 - Enables real-time visualization of device audio over USB and tuning of AFE component parameters
 - Requires [DEEPCRAFT™ Audio Enhancement Tech Pack](#)
- **Audio Front End (AFE) middleware**
 - Executes on the CM55 CPU of the PSOC™ Edge MCU
 - Includes wrapper API and control logic to interface application code with core algorithms
- **Audio voice core library (audio-voice-core)**
 - Contains the core audio algorithms
 - Optimized for PSOC™ Edge MCU (CM55 CPU + Ethos-U55)
 - Supports Arm® and LLVM compiler toolchains
 - This is a licensed deliverable from Infineon – available via [DEEPCRAFT™ Audio Pack](#)



Related code example

- DEEPCRAFT™ Audio Enhancement application
 - Demonstrates audio processing using the DEEPCRAFT™ Audio Enhancement (AE) solution
 - Application executes on the Arm® Cortex®-M55 CPU
 - Code example functionality
 - PSOC™ Edge enumerates as a USB microphone as well as a USB speaker
 - Audio captured from PDM microphones is streamed over USB Audio Class
 - Audio played from the PC over USB Audio Class is played back on the onboard speaker via I2S
 - The audio received from the kit on the PC is processed by the AE solution to remove noise, echo, and other artifacts
 - The difference can be observed by pressing the user button to compare the unprocessed audio with the processed one

Workflow – configuration to deployment

Step 1: Design phase

- Identify application audio challenges
- Determine required AE components
- Plan processing pipeline



Step 2: Configuration using AFE configurator

- Create .mtbaf project file
- Select components (HPF, AEC, BF, etc.)
- Set static parameters
- Preview pipeline in tool



Step 3: Code generation

- Generate .c/.h files using AFE Configurator
- Integrate the files with application for build



Step 4: Develop application code

- Initialize AFE middleware
- Feed audio data (10 ms frames)
- Retrieve processed audio
- Use for application (inference, playback, and so on)



Step 5: Tuning and optimization (Dynamic)

- Connect AFE Configurator to device for tuning
- Modify parameters in real-time
- Stream audio via USB for analysis
- Verify KPI metrics (ERLE, SNR, and so on)
- Save optimal configuration and integrate the final .c/.h files in application

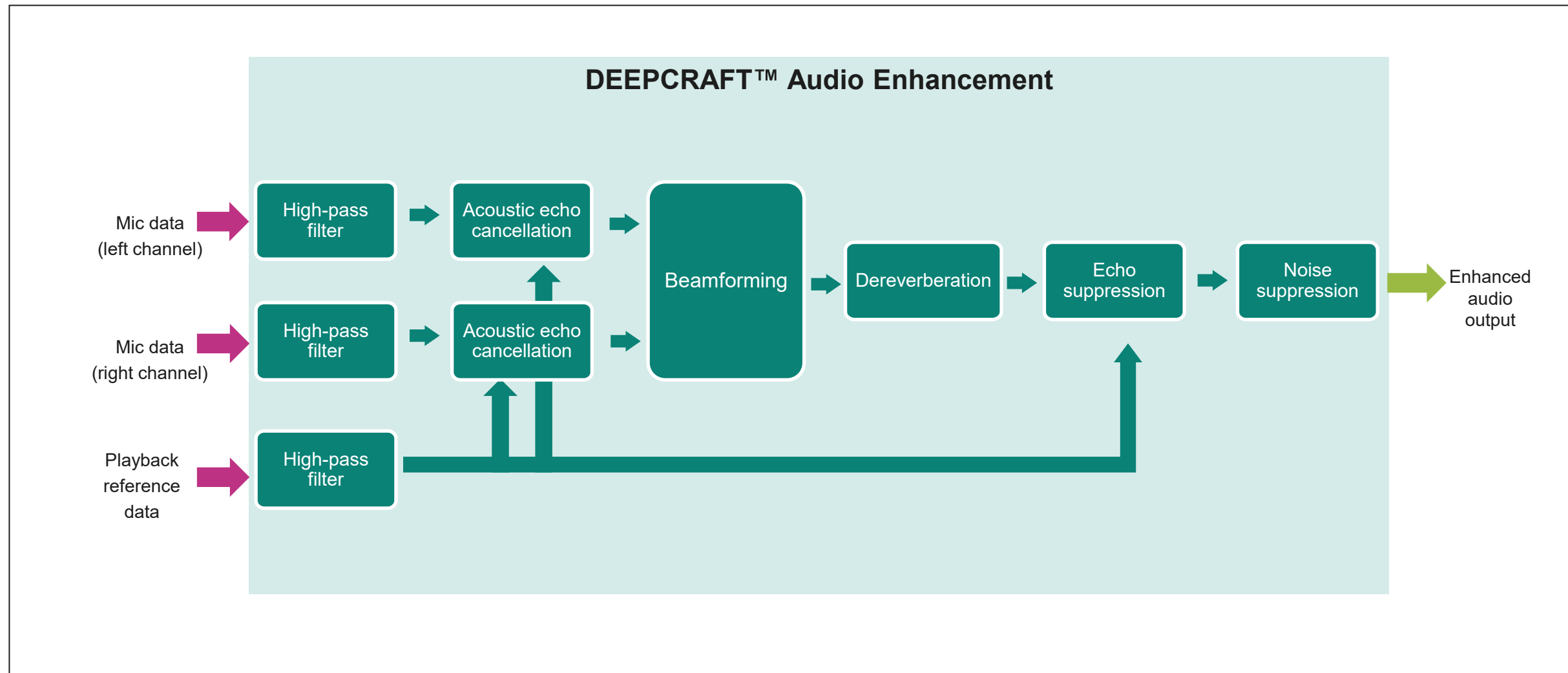


Step 6: Deployment

- Generate final firmware and program device
- Verify KPI metrics before deploying to production

Audio enhancement component – Deep drive

Pipeline – signal flow



High-pass filter (hpf)

High-pass filter (HPF)

- A simple, efficient filter that removes audio signals below a specific frequency

Purpose

- Removes unwanted, non-speech, and low-frequency noise
- **Common noise sources**
 - Wind noise
 - Handling noise (tapping or moving the device)
 - Building rumble (HVAC, traffic, and so on)
 - DC offset from the microphone

Key parameters

- **Cutoff frequency:** The frequency below which signals are attenuated
- **Default:** Approximately 100 Hz, which is suitable for most voice applications because speech content is typically concentrated above this frequency

Tuning strategy

- This is generally a **set it and forget it** component
- Minimal tuning is required. By default, 100 Hz is recommended for most voice applications

Acoustic echo cancellation (AEC)

Acoustic echo

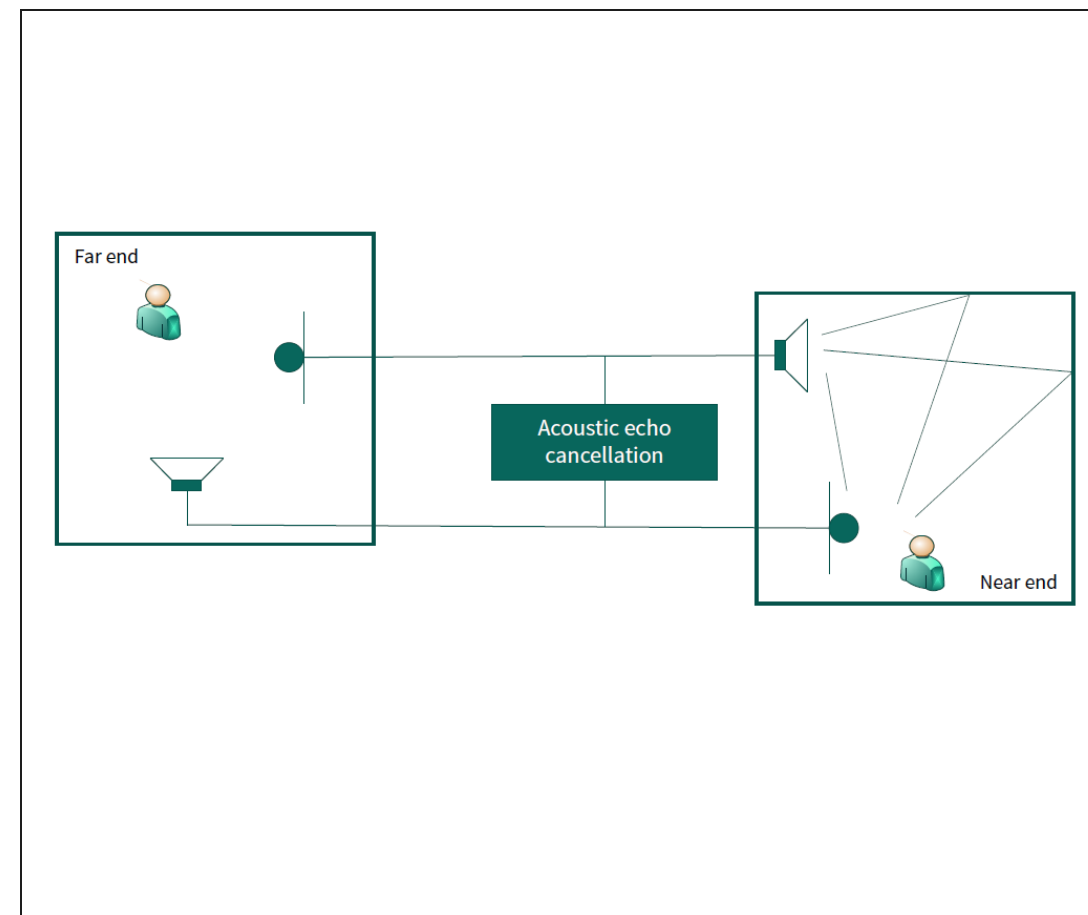
- When audio played from speakers bounces off room surfaces and gets captured by microphones, creating "echo" in the signal

Problem

- Speaker plays audio at your end
- Sound reflects off walls, ceiling, and furniture
- Microphone captures both direct speaker output and reflections
- Echo travels through an acoustic path (delay 10- 50 ms typically)
- This echo distorts the microphone audio, creating a feedback loop

Limitations of simple playback signal subtraction

- Acoustic path is not simple (multiple reflections)
- Loudspeaker has non-linear behavior
- Echo path keeps changing (people moving, objects shifting)
- White noise would result from misalignment



Acoustic echo cancellation (AEC) (contd.)

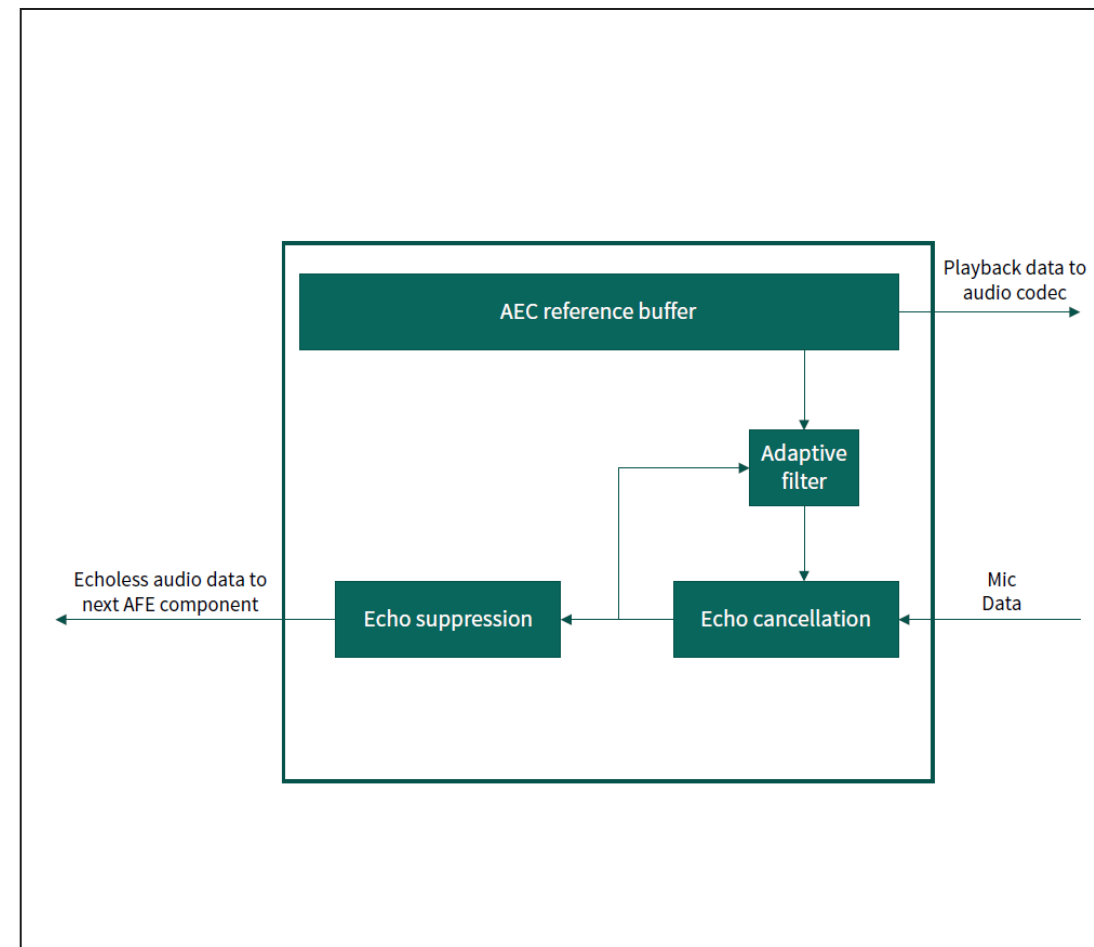
Solution: Adaptive Filtering using AEC

AEC processing overview

- **Reference signal:** Playback audio fed to AEC
- **Adaptive filter:** Continuously estimates room impulse response
- **Echo estimation:** Recreates expected echo based on reference
- **Subtraction:** Subtracts estimated echo from microphone signal
- **Echo suppression:** ML-based removal of residual echo using U55

Is AEC critical ?

- Voice calling with background speaker
- Voice assistant running while playing music/audio
- Simple voice capture (no playback)



Acoustic echo cancellation (AEC) parameters

Bulk delay (ms)

Time between speaker audio output and microphone capture

- **Includes:** Acoustic propagation + digital buffering delays
- **Why critical:** AEC needs to search for echo at correct time
- **How to measure:** AFE Configurator has calibration function (plays chirp)
- **Typical:** 10-50 ms

Tail length (ms)

How long to track echo (impulse response duration)

- **Typical:** 10-64 ms
- Shorter = faster adaptation, less computation
- Longer = better tracking in reverberant rooms
- **Trade-off:** Balance latency vs accuracy

Echo suppression aggressiveness

How hard to suppress residual echo

- **Range:** 0-100% or 0-20 dB
- **Low:** Leaves some echo, cleaner speech
- **High:** Removes all echo, possible speech distortion
- **Recommendation:** Medium (70-80%) for a balanced result

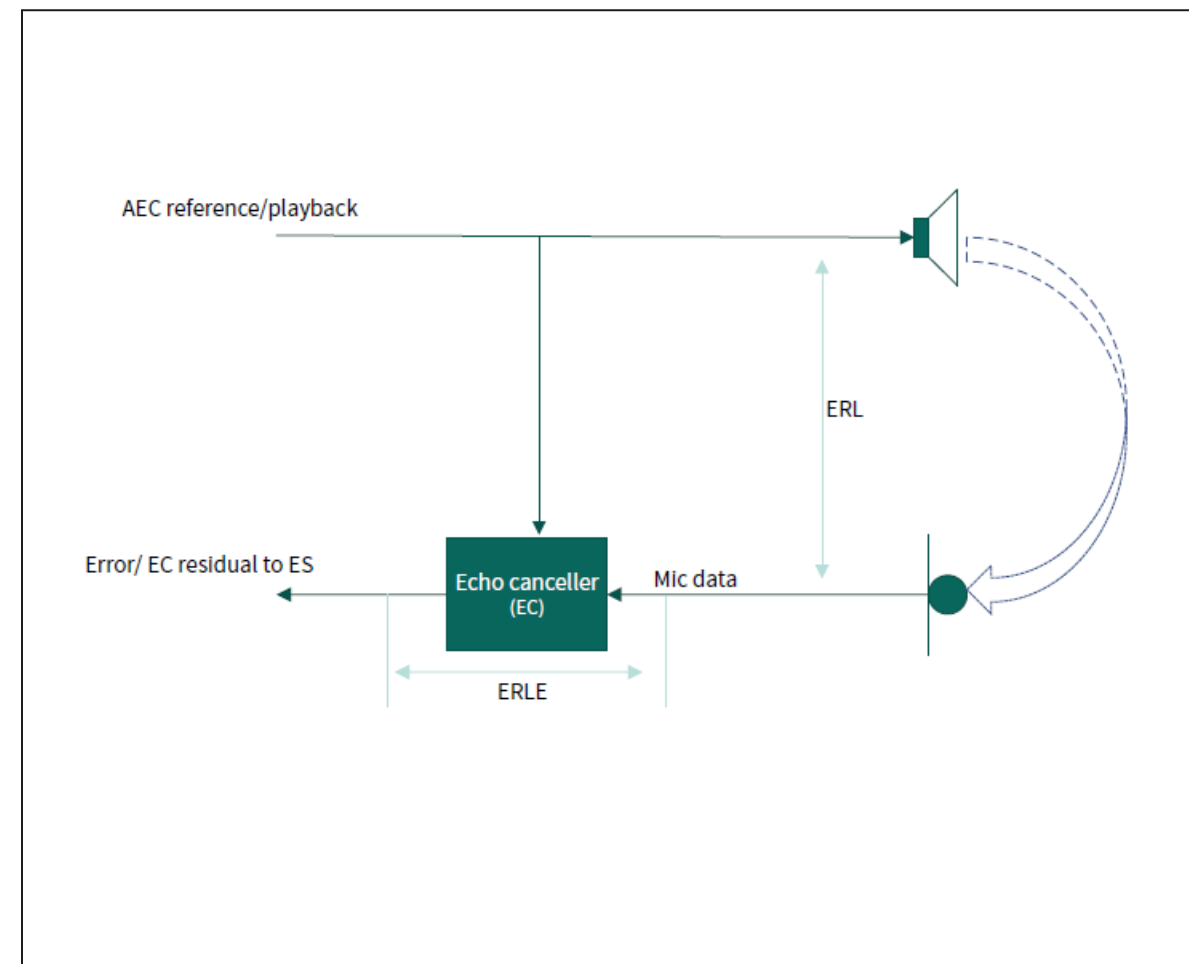
Acoustic echo cancellation (AEC) – Performance metrics

Echo return loss (ERL)

- Natural room attenuation
- **Formula:** $ERL = 10 \times \log_{10} \frac{P_{speaker}}{P_{residual\ echo}}$
- **Typical:** 10-20 dB (room just naturally attenuates echo)
- Depends on: Room size, materials, speaker/mic placement

Echo return loss enhancement (ERLE)

- Improvement from AEC algorithm
- **Formula:** $ERLE = 10 \times \log_{10} \frac{P_{microphone}}{P_{residual\ after\ AEC}}$
- **Target:** >15 dB (good), >25 dB (excellent)
- If ERLE < 10 dB: Something is wrong (check calibration, placement, and so on)



Beamforming (BF)

Beamforming

- A signal processing technique that uses multiple microphones to create directional sensitivity
- A signal processing technique that uses multiple microphones to create directional sensitivity

Importance of Beamforming

- A single microphone captures sound from all directions (omnidirectional pickup)
- In noisy environments, unwanted sounds are captured along with the speaker's voice
- Beamforming enhances audio quality by focusing on the speaker's direction and attenuating sounds arriving from other directions
- Crucial for multi-speaker environments

Working of Beamforming

1. A microphone array (typically 2 mics, 20-100 mm apart) captures sound
2. Sound reaches each microphone at slightly different times due to differences in arrival distance.
3. The algorithm analyzes these time and phase differences to estimate the **Direction of Arrival (DOA)**
4. Steering vectors align microphone signals for constructive interference in the desired direction
5. **Result:** Higher sensitivity in the desired direction; audio from other directions is suppressed

Beamforming (BF) – Usage

Using beamforming

- Far-field voice use cases
- Noisy environments (office, outdoor)
- Single-speaker focus is required
- Do not use in cases of extremely close range where beams overlap

Beamforming (BF) – parameters

Microphone distance (mm)

Physical spacing between microphones

- **Typical:** 40-50 mm
- **Critical:** Must be exact (± 5 mm tolerance) or DOA estimation fails
- Must match configuration exactly

Number of beams

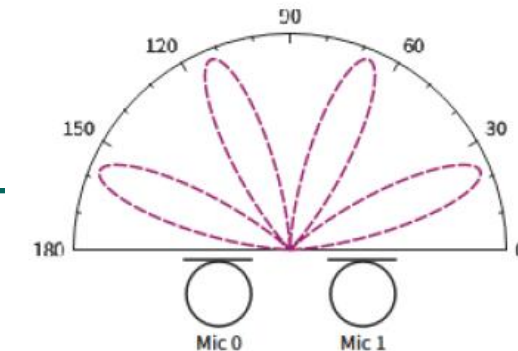
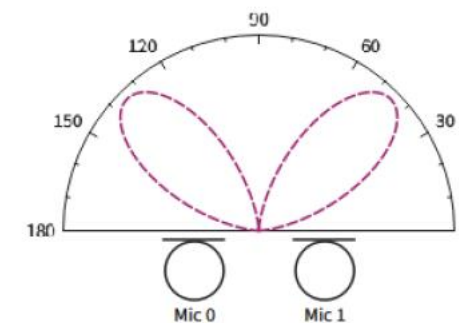
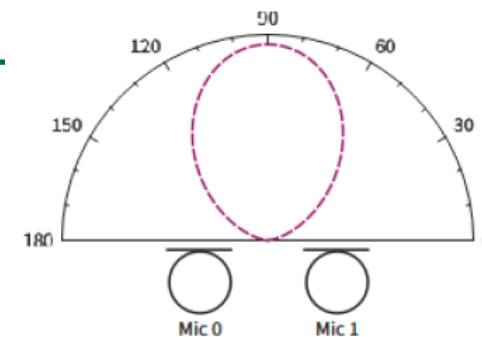
- **One beam:** Tight focus on a single direction (best for a single speaker)
- **Four beams:** Omnidirectional with null points
- **More beams** = less per-beam gain but broader coverage

Angle range (degree)

- **Front 0-180°:** Cover everything ahead
- **Narrower 45-135°:** Only sides and front, ignore rear
- Trade-off: Narrower = more focused, but less flexibility

Interference canceller aggressiveness

- **Low:** Minimum artifacts, some background noise
- **Medium:** Good balance
- **High:** Maximum noise suppression, possible speech distortion



Dereverberation (DRVB)

Reverberation

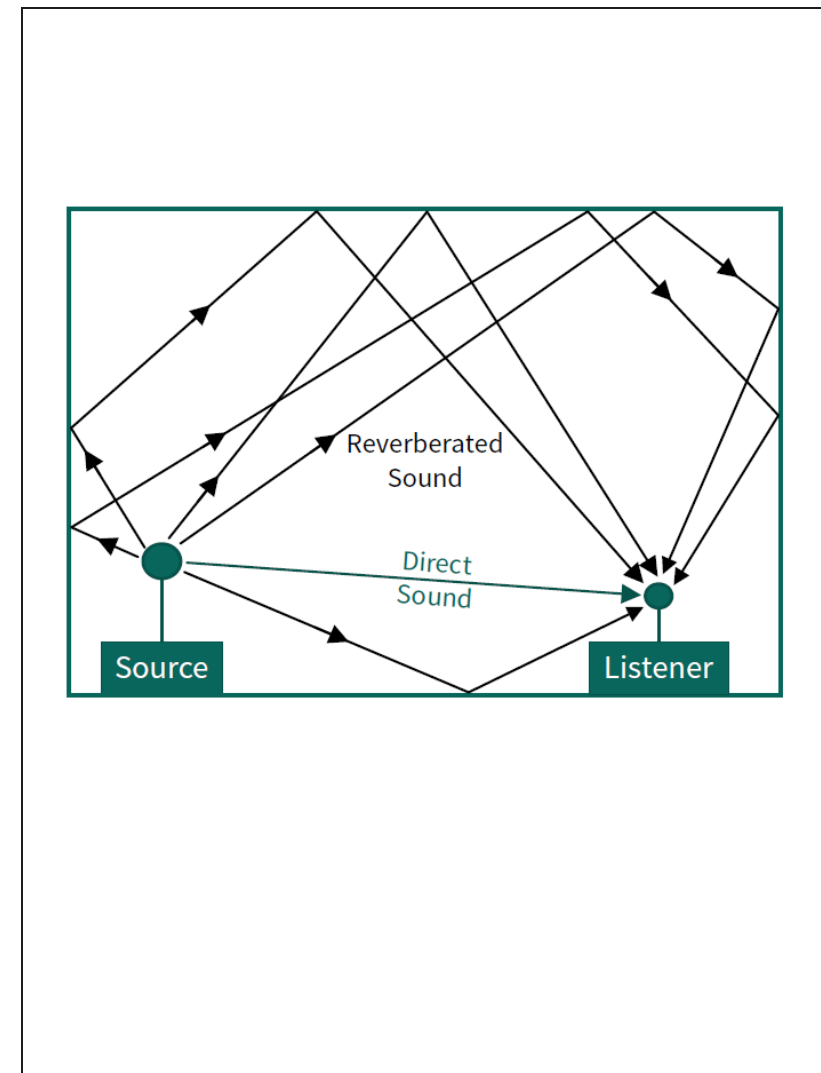
- Sound reflections that continue to persist after the original sound source has stopped
- Characterized by RT60, the time required for the sound level to decay by 60 dB
- Causes speech to sound distant, muffled, or "echoey"
- Can significantly reduce speech intelligibility and voice recognition accuracy

Impact of Dereverberation

- Reduces recognized echo tail effects, improving subsequent components like echo suppression
- Particularly important for ASR (speech recognition) in reverberant environments
- Modest CPU cost

Key metric: RT60 (reverberation time)

- How quickly sound decays after silence
- **< 0.3s**: Dry room (treated, carpeted)
- **0.5-1.0s**: Typical office
- **> 1.0s**: Very reverberant (like gym, church, and so on)
- Higher RT60 → Need more dereverberation



Noise suppression (NS)

Problem

- **Background noise:** Traffic, HVAC, crowd, and music
- Degrades voice quality and intelligibility
- Reduces speech recognition accuracy significantly

Working of noise suppression

- Analyse Audio (STFT)
- Separate magnitude and phase
- Generate suppression mask with ML inference on U55 NPU
- Apply this mask
- Reconstruct audio (inverse STFT) to complete denoising

Noise suppression (NS) – Parameters

Aggressiveness (0-20 dB)

- **Low (0 dB to 5 dB):** Minimal distortion, less noise reduction
- **Medium (10 dB):** Balanced quality/noise reduction
- **High (15 dB to 20 dB):** Aggressive, may introduce artifacts

Trade-off

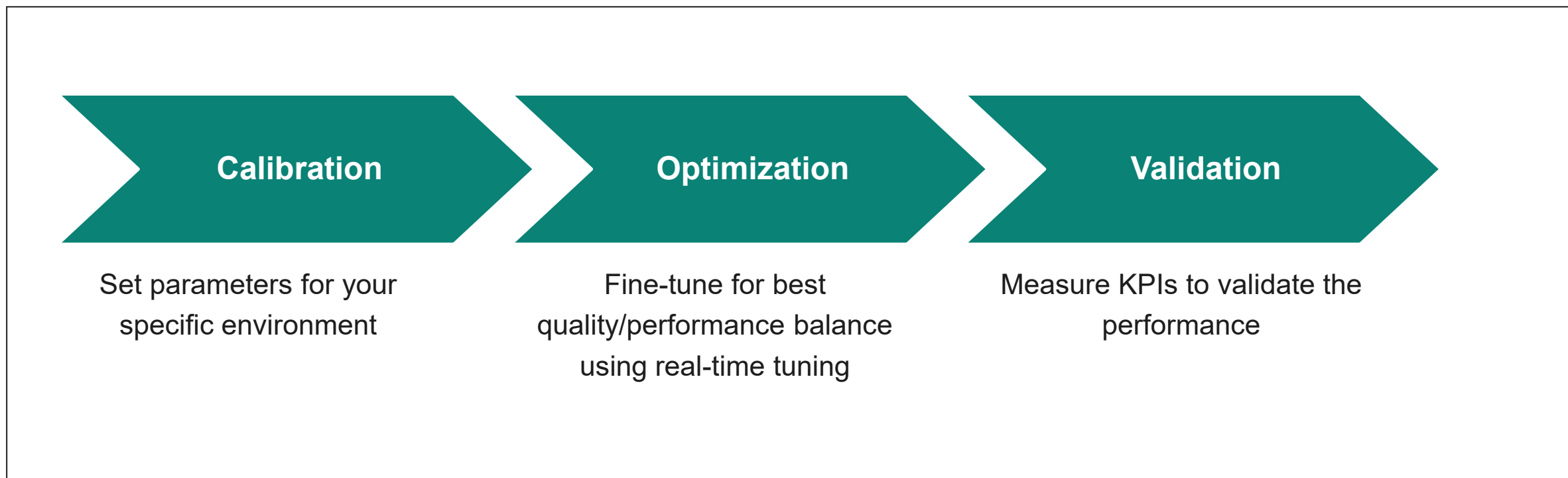
- **More aggressive = more noise removed but potential speech distortion**
 - Speech distortion may not affect speech recognition systems, but it is very critical for applications like voice calling
- **Less aggressive = cleaner sound but residual noise audible**

Dereverberation and Noise suppression

Component	Purpose	Input	Output	Machine learning (ML)	CPU	Applications
Dereverberation	Remove room reflections	Single channel	Direct sound	No	Low	Reverberant rooms
Echo suppression	Residual echo removal	Single channel	Clean echo-free	Yes	U55	After AEC
Noise suppression	Background noise reduction	Single channel	Clean audio	Yes	U55	Any noisy environment

AFE tuning strategy and best practices

Audio enhancement tuning – three-step approach



AFE tuning strategy and best practices (contd.)

High pass filter (HPF)

- Pre-configured, minimal tuning needed
- **Default cuto ff:** ~100 Hz (removes rumble, handling noise)

Echo cancellation (AEC)

- **Critical calibration:** Bulk delay measurement (use calibration tool)
- **Aggressiveness:** Match environment (quiet room vs conference hall)
- **Common issue:** Incomplete echo cancellation = wrong bulk delay

Beamforming (BF)

- **Microphone spacing:** Must match physical spacing (20-100 mm)
- **Angle range:** Match expected speaker directions
- **Number of beams:** Typically, 1 unless multiple speaker zones are needed

Dereverberation (DRVB)

- No user tuning needed
- Works best in highly reverberant environments
- Recommended to be enabled

Noise suppression (NS)

- Most subjective parameter depends on application; test with multiple talkers/noise environments
- Start aggressive, then reduce until optimal noise suppression-naturalness balance is achieved
- Speech recognition can tolerate more aggressive settings than recording/teleconference

AFE tuning strategy and best practices (contd.)

Tuning workflow in Lab 2

1. Enable components one at a time
2. Listen to the impact of each
3. Measure ERLE/SNR if tools are available
4. Combine components, retest
5. Iterate until satisfied with quality

Common mistakes to avoid

- Enabling all components without understanding impact
- Setting noise suppression too high (creates artifacts)
- Skipping AEC calibration (worst echo cancellation results)
- Not testing in a representative acoustic environment
- Always test with real speech, not test signals
- Measure before or after for objective comparison

Audio enhancement – Key performance indicators (KPIs)

Echo return loss enhancement (ERLE)

- **Definition:** Improvement in echo reduction compared to baseline
- **Measurement:** $10 \times \log_{10} \frac{P_{\text{microphone}}}{P_{\text{residual echo after AEC}}}$
- **Unit:** Decibels (dB)
- **Target:** 15-30 dB typically, >25 dB excellent
- **What it means:** How much AEC reduces echo
- **Measurement condition:** Quiet room, no background noise

Perceptual evaluation of speech quality (PESQ)

- **Definition:** Subjective listening test score predicted by algorithm
- **Scale:** -0.5 to 4.5
 - 4-5: Excellent (imperceptible degradation)
 - 3-4: Good (slightly perceptible, not annoying)
 - 2-3: Fair (perceptible, slightly annoying)
 - 1-2: Poor (very annoying)
 - <1: Bad (communication difficult)
- **Benchmark:** Most applications target PESQ > 3.5
- **Practical:** Compare before/after AE processing

Audio enhancement – Key performance indicators (KPIs)(contd.)

Signal-to-Noise ratio (SNR)

- **Definition:** Ratio of desired signal power to noise power
- **Unit:** Decibels (dB)
- **Calculation:** $10 \times \log_{10} \frac{P_{signal}}{P_{noise}}$
- **Benchmark:**
 - >20 dB: Noise barely noticeable
 - 10-20 dB: Noise present but manageable
 - <10 dB: Noise very apparent
 - Negative: Noise louder than signal

Other relevant metrics

- **Latency:** Time from input to output (target <100 ms for teleconferencing)
- **MCPS:** Million cycles per second (CPU computational cost)
- **Memory:** Code and Data size (Scratch + persistent memory requirements)

Table of contents

1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement

What will you learn?

- How to use the DEEPCRAFT™ Audio Enhancement pipeline on the PSOC™ Edge CM55 core
- How to use “Functional” mode to observe real-time noise suppression
- How to use the AFE configurator tool in “Tuning” mode to dynamically modify AE parameters without recompiling
- Learn two methods for visualizing the multi-channel debug audio stream

Table of contents

1	Prerequisites	2
2	Audio input and output methods using PSOC™ Edge MCUs	6
3	Lab 1: Understanding PDM microphone configuration – 16-bit vs 24-bit	19
4	DEEPCRAFT™ Audio Enhancement on PSOC™ Edge MCUs	21
5	Lab 2: Hands-on with DEEPCRAFT™ Audio Enhancement	47
6	DEEPCRAFT™ Voice Assistant on PSOC™ Edge MCUs	49

DEEPCRAFT™ Voice Assistant

Overview

- DEEPCRAFT™ Voice Assistant is a platform for creating and deploying customizable voice recognition solutions:
 - **Wake word detection:** Custom wake words (not just predefined phrases)
 - **Command recognition:** Define your own voice commands
 - **Intent mapping:** Actions triggered by voice
 - **On-device inference:** All processing happens locally (private)
- **Architecture**
 - **Cloud-Dependent Approach:**
 - Speech → Cloud → Processing → Response
 - Latency, privacy, and connectivity
 - **DEEPCRAFT™ Voice Assistant (Local, edge-optimized):**
 - Speech → Local Model → Inference → Response
 - Fast, Private, always-on, and low-power

Getting started

To get started with the Audio Enhancement, see the **DEEPCRAFT™ Voice Assistant development flow** section in the [PSOC™ Developer Journey Trainings](#).

1. Starting a Voice Assistant Design

This video explains how to log into the Voice Assistant Designer, provides an overview of the User Interface and explains wake words, text, variables and intent when building a voice model.

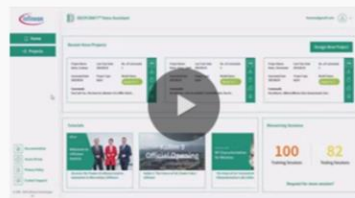
[Open resource](#)



[Open in media player](#)

2. Creating a basic voice model using Voice Assistant Cloud Tool

Create a new project. Build a Simple Command Structure, Generate a Model from your Project.



[Open in media player](#)

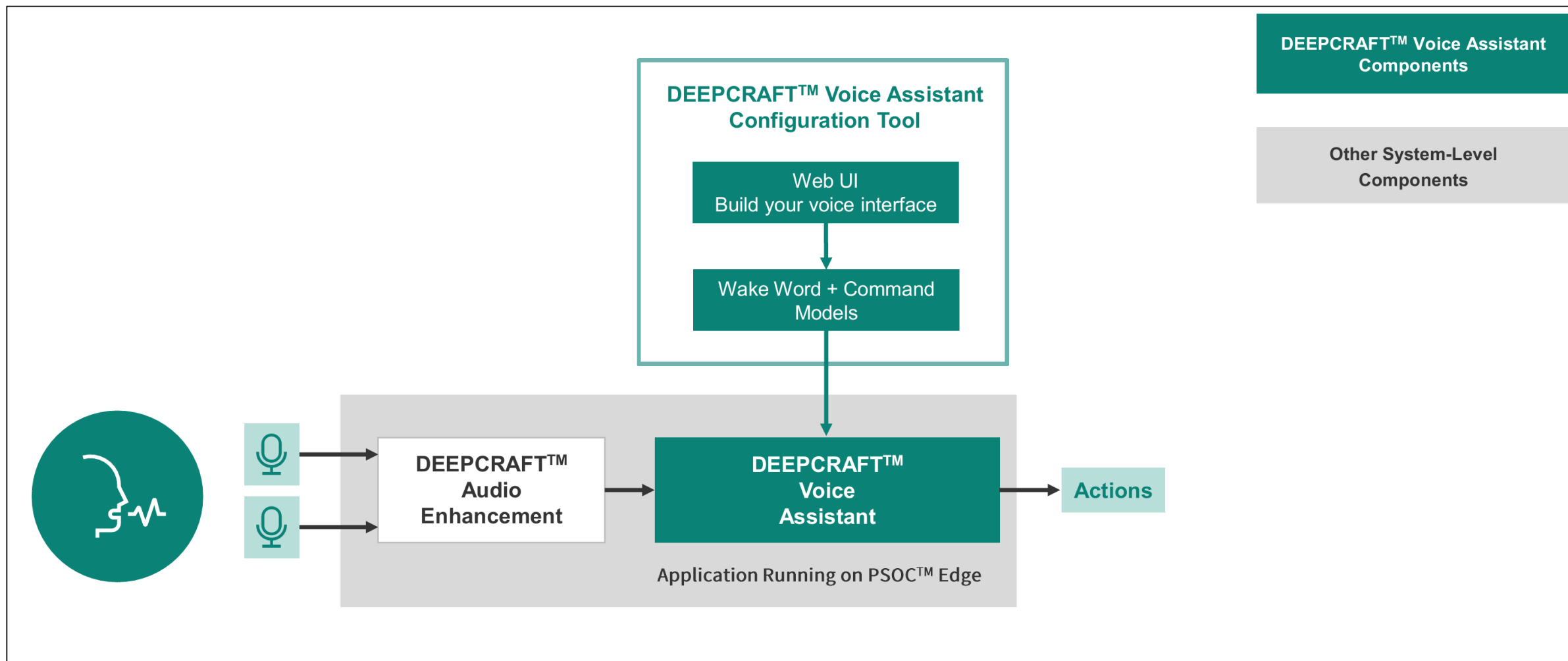
3. Testing Your Model in the Cloud

Testing your model in the cloud.



[Open in media player](#)

System block diagram



Related code examples

– DEEPCRAFT™ Voice Assistant deployment

- Demonstrates deployment of DEEPCRAFT™ Voice Assistant (VA) models to detect wake words and commands.
- Application executes on the CM55 CPU
- Includes DEEPCRAFT™ Audio Enhancement in the audio pipeline for noise suppression
- Simplest CE to get started with voice assistant and deploy your own models

– Mains-powered local voice

- Superset example that includes:
 - DEEPCRAFT™ Voice Assistant
 - DEEPCRAFT™ Audio Enhancement – Complete pipeline
 - PDM / USB Audio Input
 - I2S / USB Audio Output
 - 2D Graphics Output for Music Player use-case
 - Built-in profiler for MCPS measurement for various components
 - Clocks and memory optimized for performance
- Well-suited to build complex applications and optimize for performance

Model deployment workflow

Step 1: Project creation

- Define wake word
- Define command intents
- Define command variations

Step 3: Cloud-based testing

- Upload test audio samples
- Review failure cases

Step 5: Device deployment

- Copy models to application
- Build and flash
- Test in real environment
- Iterate if needed

Step 2: Model generation (in cloud)

- Generate the model by initiating model training and optimization for the U55 neural accelerator
- **Processing time:** Mode-dependent

Step 4: Download optimized models

- Download model zip file
- Contains WW and command model files

Wake word detection and command recognition

Wake word detection

- Input: Continuous 16 kHz audio
- Output: Binary (wake word detected: yes/no)
- Accuracy depends on the following:
 - Speech quality (SNR)
 - Distance (1-6 feet practical)
 - Speaker accent/gender (model trained on diverse)
 - Background noise type
 - Wake word design (clear vs ambiguous)

Command recognition

- Input: Audio segment post WW (typically 0.5-5 sec)
- Output: Intent
- Accuracy depends on the following:
 - Number of commands (fewer = higher accuracy)
 - Command similarity (more distinct = higher accuracy)
 - Audio quality (SNR, reverberation)
 - Training data diversity
 - Audio enhancement preprocessing

Key performance index (KPI)

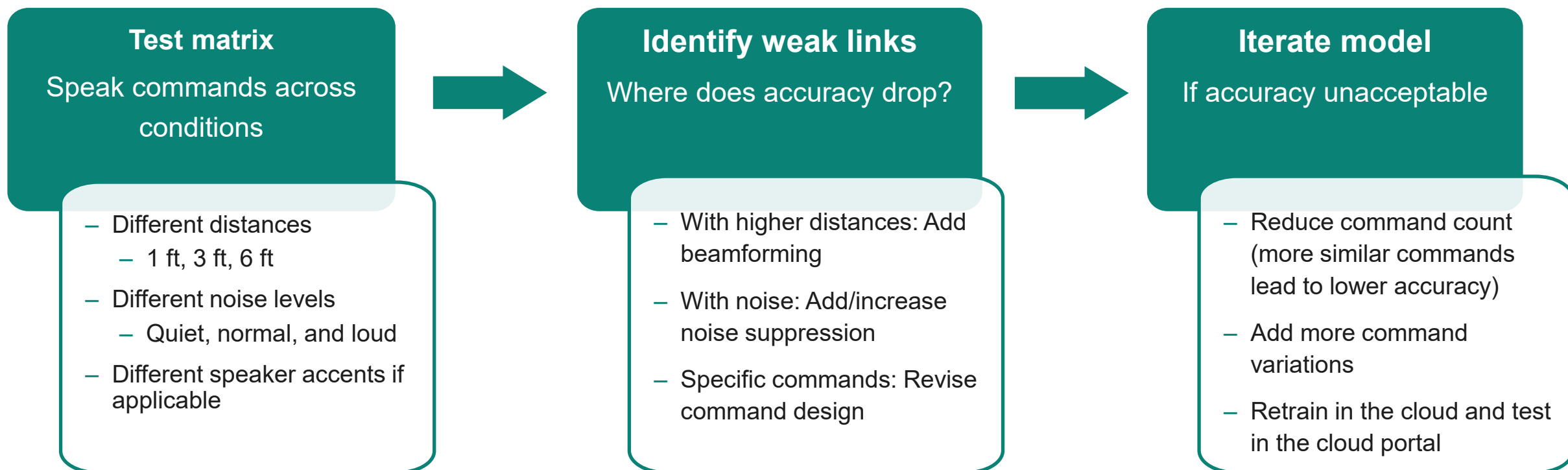
Wake-up accuracy

- **Definition:** % of spoken wakewords that get detected correctly by the device
- **Measurement:** Speak wakeword 'n' number of times, count detections from the device
- **Target:** 95%+ (practically acceptable)
- **Factors affecting:**
 - Speaking distance (1 ft optimal, degrades at 6+ ft)
 - Background noise level (SNR)
 - Speaker accent/language
 - Device microphone quality
 - For example, "Okay Infineon", 1 ft, quiet room → 98% Wake-Up Accuracy typical

Command understanding accuracy

- **Definition:** % of spoken commands correctly mapped to intent
- **Measurement:** After wake word, speak command 'n' number of times, count correct intents from the device
- **Target:** 90%+ in typical conditions
- **Factors affecting:**
 - Command similarity (close commands → lower accuracy)
 - Audio quality (after AE processing)
 - SNR of environment
 - Speaker familiarity with phrasing

KPI optimization



Best practices

- Ensure phonetically distinct commands
 - "Play" vs "Pause" – Good
 - "Play" vs "Pay" – May yield poor results
- Multiple variations per command (handles natural speech)
 - "Raise Volume" and "I can't hear you" – both map to *increase_volume* intent
- Use minimal set of commands for optimal performance (more = lower accuracy)
- **Design intent-based commands:** Think intent, not exact text. Map multiple phrasings to the same intent

Wakeword characteristics

- Optimal: 2-3 syllables for lower false positives ("OK Infineon", "Hey Siri", "Okay Google")
- Avoid: Single syllable ("Alex") – triggers accidentally
- Avoid: Homophonic with common words ("OK" can be in normal speech)
- Avoid homophones or similar-sounding commands
 - "Increase" vs "Decrease" – similar-sounding commands – may yield poor results
- Don't include wakeword as a command prefix

Testing recommendations

Test with different speakers

- Different genders, accents, ages
- Training data diversity is crucial

Test at various distances

- 1 ft, 3 ft, and 6 ft
- Understand effective range

Test with different background noise

- Quiet
- Busy office with background
- Kitchen noise
- Outdoor with wind/traffic
- Device playing music simultaneously

Measure end-to-end latency

- Should be <300 ms for a natural feel
- Ensure overall latency doesn't cross ~800ms to avoid sluggish and unresponsive user experience

Audio Enhancement in voice pipeline

To enable AE components

- **Always:** Noise Suppression (improves accuracy in noisy environments)
- **Usually:** AEC (if any speaker output happens)
- **Recommended:** Beamforming (far-field robustness)
- **Optional:** Dereverberation (in very reverberant rooms)
- **Setup:** Enable AE before VA inference (preprocessing)

Trade-offs

- More AE = Better noise robustness but higher latency
- Careful tuning is critical for natural-sounding interaction
- Audio quality impacts user perception of accuracy

Deployment guidelines

- Profile system before deployment
 - Measure MCPS at each stage
 - Ensure headroom for future features (20%+)
- Design for degraded audio
 - Plan for worst-case (low SNRs scenarios)
 - Don't assume clean input
- Log performance metrics
 - Track Wake-Up Accuracy and Command Understanding Accuracy in the field
 - Detect acoustic environment changes
 - Enable future optimization
- Provide user feedback
 - Visual (LED, display) or Audio (beep, confirmation) cues when listening and command recognition
 - Clear failure messages

References

Documentation

- [DEEPCRAFT™ Audio Enhancement user guide](#)
- [DEEPCRAFT™ Voice Assistant user guide](#)
- [Voice and audio firmware components guide](#)
- [PSOC™ Edge \(E8x2, E8x3, E8x5, E8x6\) architecture reference manual](#)
- [PSOC™ Edge E8x2, E8x3, E8x5, E8x6 Consumer datasheet](#)

Code examples (GitHub)

- [mtb-example-psoc-edge-ae-application](#)
- [mtb-example-psoc-edge-voice-assistant-deploy](#)
- [mtb-example-psoc-edge-mains-powered-local-voice](#)
- [mtb-example-psoc-edge-pdm-pcm](#)
- [mtb-example-psoc-edge-i2s](#)

Training resources

- [PSOC™ Edge trainings](#)
- [AE video tutorial](#)
- [Voice Assistant cloud platform tutorial](#)

Revision history

Document revision	Date	Description of change
**	2026-08-14	Initial release.

